Week 05_Gating sets

Macarena López-Mayorga

2026-04-23

```
Folder <- file.path("C:/Users/Macarena/CytometryInR/course/05_GatingSets/data")
fcs_files <- list.files(Folder, pattern=".fcs", full.names=TRUE)
fcs_files
```

[1] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF052_00_Ctrl.
[2] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF052_00_SEB
[3] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF100_00_Ctrl.
[4] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF100_00_SEB
[5] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF179_00_Ctrl.
[6] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF179_00_SEB

We are going to name one of them as "fcs-files[x]". In our case, the first file

```
fcs_files[1]
```

[1] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF052_00_Ctrl.

```
library(flowCore)
```

```
flowFrame <- read.FCS(filename=fcs_files[1], truncate_max_range = FALSE,
 transformation = FALSE)
flowFrame
```

flowFrame object '2025_07_26_AB_02_INF052_00_Ctrl.fcs' with 10000 cells and 43 observables: name desc range minRange maxRange $P1TimeNA8967450878.809P2$ SSC-W NA 4194304 0 4194303.000 $P3SSC - HNA419430404194303.000P4$ SSC-A NA 4194304 0 4194303.000 $P5FSC - WNA419430404194303.000$.....P39 APC-R700-A CD107a 4194304 -111 4192506 $P40ZombieNIR - AViability4194304 - 1114192506P41$ APC-Fire 750-A CD27 4194304 -111 4192506 $P42APC - Fire810 - ACCR74194304 - 1114192506P43$ AF-A NA 4194304 -111 4194303 472 keywords are stored in the 'description' slot

```r
str(fcs_files)
```

chr [1:6] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF052_00_
…

All of them are character class.

If we want to read multiple fcs files at once, we will need another function:

::: {.cell}

```r
flowSet <- read.flowSet(files=fcs_files, truncate_max_range = FALSE, transformation = FALSE)
flowSet
```

:::

A flowSet with 6 experiments.

column names(43): Time SSC-W … APC-Fire 810-A AF-A

Also, we can select those fcs files that we can specifically read:

```r
read.flowSet(files=fcs_files[c(1, 3:4)], truncate_max_range = FALSE, transformation = FALSE)
```

A flowSet with 3 experiments.

column names(43): Time SSC-W … APC-Fire 810-A AF-A

we can see that read.flowSet() has created a "flowSet" class object.

```r
class(flowSet)
```

[1] "flowSet" attr(,"package") [1] "flowCore"

In the right secondary side bar, we can see that "flowSet" is another Bioconductor style S4-type object that within its frame slot contains individual "flowFrames". If instead of class() we had used str(), we would have seen a similar output ton what we see in the Variables panel.

```r
str(flowSet)
```

Formal class 'flowSet' [package "flowCore"] with 2 slots ..@ frames :<environment: 0x000001fb5d4cf5a8> ..@ phenoData:Formal class 'AnnotatedDataFrame' [package "Biobase"] with 4 slots .. .. ..@ varMetadata :'data.frame': 1 obs. of 1 variable: .. ..$ labelDescription: chr "Name" .. .. ..@ data :'data.frame': 6 obs. of 1 variable: .. ..$ name: 'AsIs' chr [1:6] "2025_07_26_AB_02_INF052_00_Ctrl.fcs" "2025_07_26_AB_02_INF052_00_SEB.fcs"

“2025_07_26_AB_02_INF100_00_Ctrl.fcs” “2025_07_26_AB_02_INF100_00_SEB.fcs” … .. .. ..@ dimLabels : chr [1:2] “rowNames” “columnNames” .. .. ..@ .___classVersion___:Formal class ‘Versions’ [package “Biobase”] with 1 slot .. ..@ .Data:List of 1 .. ..$: int [1:3] 1 1 0 .. ..$ names: chr “AnnotatedDataFrame”

REMINDER: While not today’s focus, remember we could access individual components inside the flowSet using the @ accessors covered during Week3.

One consideration of this method is the contents of your .fcs files are read into your computer’s random access memory (RAM). While for individual .fcs files or small experiments this will not present a problem for most modern computers, when working with large spectral flow cytometry files containing millions of events (or trying to analyze many .fcs files at once), you may encounter situations where you can quickly exceed your computers available RAM. To build some contextual understanding of the problem, let’s learn how to check how much memory is being used by our individual variables/objects within our R session. We will primarily use the lobstr R packages obj_size() function, as it better handles evaluating complicated objects than base R’s object.size() function.

::: {.cell}

```
object.size(flowFrame)
```

:::

3570976 bytes

::: {.cell}

```
install.packages("lobstr")
```

:::

::: {.cell}

```
 library(lobstr)
obj_size(flowFrame)
```

:::

3.53 MB FlowSet object contains multiples flowFrame objects, so the size should be bigger:

::: {.cell}

```
obj_size(flowSet)
```

:::

20.99 MB

If we were curious how much memory total we are using within R at the current moment, we can check using the mem_used() function:

::: {.cell}

```
mem_used()
```

:::

141.95 MB

Ultimately, how many .fcs files you are able to read in and interact with before running out of available RAM memory space will be dictated by your individual computers hardware configuration. There are various ways you can check programmatically how much RAM your computer has available, although the specific functions will vary depending on your computers operating system, since they often involve system-level code outside R. Using the ps R package's ps_system_memory() function is one of the easier ways for Windows users.

To simplify the process, here is an additional example of where a conditional can prove useful, allowing us to check in an operating system specific manner. It takes the output of the Sys.info() function, namely the "sysname" argument and then retrieves the relavent function.

```
OperatingSystem <- Sys.info()[["sysname"]]
if (OperatingSystem == "Windows")
Memory <- ps::ps_system_memory()
message("Total GB ", round(Memory$total / 1024^3, 2))
message("Free GB ", round(Memory$free / 1024^3, 2))
```

Total GB 15.78 Free GB 7.69

```
install.packages("ps")
```

```
library(ps)
Memory <- ps::ps_system_memory()
message("Total GB ", round(Memory$total / 1024^3, 2))
```

Total GB 15.78

```r
message("Free GB ", round(Memory$free / 1024^3, 2))
```

Free GB 7.83

In addition to the flowCore R package, additional flow cytometry infrastructure support is provided by the flowWorkspace package. Instead of the reading all the .fcs files contents into active RAM, flowWorkspace reduces the memory overhead by using "pointers" to interact with the object in it's current storage location (either on your harddrive, SSD, etc.), only reading in components to RAM as needed.

```r
library(flowWorkspace)
```

As part of improvements to flowWorkspace, some behavior of GatingSet objects has changed. For details, please read the section titled "The cytoframe and cytoset classes" in the package vignette:

vignette("flowWorkspace-Introduction", "flowWorkspace")

::: {.cell}

```r
cytoframe <- load_cytoframe_from_fcs(fcs_files[1], truncate_max_range = FALSE,
 transformation = FALSE)
cytoframe
```

:::

cytoframe object '2025_07_26_AB_02_INF052_00_Ctrl.fcs' with 10000 cells and 43 observables: name desc range minRange maxRange $P1TimeNA878.8090878.809P2$ SSC-W NA 4194303.000 0 4194303.000 $P3SSC-HNA4194303.00004194303.000P4$ SSC-A NA 4194303.000 0 4194303.000 $P5FSC-WNA4194303.00004194303.000$.....P39 APC-R700-A CD107a 4192506 -111 4192506 $P40ZombieNIR-AViability4192506-1114192506P41$ APC-Fire 750-A CD27 4192506 -111 4192506 $P42APC-Fire810-ACCR74192506-1114192506P43$ AF-A NA 4194303 -111 4194303 472 keywords are stored in the 'description' slot row names(0):

```r
class(cytoframe)
```

[1] "cytoframe" attr(,"package") [1] "flowWorkspace"

Which also still errors out when not given a scalar object

```r
load_cytoframe_from_fcs(fcs_files, truncate_max_range = FALSE, transformation = FALSE)
```

Error: ! Expected string vector of length 1

We can use Cytoset to handle multiples fcs files:

::: {.cell}

```r
cytoset <- load_cytoset_from_fcs(fcs_files, truncate_max_range = FALSE,
transformation = FALSE)
cytoset
```

:::

A cytoset with 6 samples.

column names: Time, SSC-W, SSC-H, SSC-A, FSC-W, FSC-H, FSC-A, SSC-B-W, SSC-B-H, SSC-B-A, BUV395-A, BUV563-A, BUV615-A, BUV661-A, BUV737-A, BUV805-A, Pacific Blue-A, BV480-A, BV570-A, BV605-A, BV650-A, BV711-A, BV750-A, BV786-A, Alexa Fluor 488-A, Spark Blue 550-A, Spark Blue 574-A, RB613-A, RB705-A, RB780-A, PE-A, PE-Dazzle594-A, PE-Cy5-A, PE-Fire 700-A, PE-Fire 744-A, PE-Vio770-A, APC-A, Alexa Fluor 647-A, APC-R700-A, Zombie NIR-A, APC-Fire 750-A, APC-Fire 810-A, AF-A

```r
class(cytoset)
```

[1] "cytoset" attr(,"package") [1] "flowWorkspace"

Unlike "flowFrame" and "flowSet", when we run str(), for "cytoframe" and "cytoset" objects we don't get back quite as much information.

```r
str(cytoframe)
```

Formal class 'cytoframe' [package "flowWorkspace"] with 5 slots ..@ pointer : ..@ use.exprs : logi TRUE ..@ exprs : num[0 , 0] ..@ parameters :Formal class 'AnnotatedDataFrame' [package "Biobase"] with 4 slots .. .. ..@ varMetadata :'data.frame': 5 obs. of 1 variable: .. ..$ labelDescription: chr [1:5] "Name of Parameter" "Description of Parameter" "Range of Parameter" "Minimum Parameter Value after Transformation" … .. .. ..@ data :'data.frame': 0 obs. of 5 variables: .. ..$ name : chr(0) .. ..$ description: chr(0) .. ..$ range : num(0) .. ..$ minRange : num(0) .. ..$ maxRange : num(0) .. .. ..@ dimLabels : chr [1:2] "rowNames" "columnNames" .. .. ..@ .___classVersion___:Formal class 'Versions' [package "Biobase"] with 1 slot .. ..@ .Data:List of 1 .. ..$: int [1:3] 1 1 0 .. ..$ names: chr "AnnotatedDataFrame" ..@ description:List of 1 .. ..$ note: chr "empty"

::: {.cell}

```
str(cytoset)
```

:::

Formal class ‘cytoset’ [package “flowWorkspace”] with 3 slots ..@ pointer : ..@ frames :<environment: 0x000001e3cf33dee8> ..@ phenoData:Formal class ‘AnnotatedDataFrame’ [package “Biobase”] with 4 slots .. .. ..@ varMetadata :‘data.frame’: 0 obs. of 1 variable: .. ..$ labelDescription: chr(0) .. .. ..@ data :‘data.frame’: 0 obs. of 0 variables .. .. ..@ dimLabels : chr [1:2] “rowNames” “columnNames” .. .. ..@ .___classVersion___:Formal class ‘Versions’ [package “Biobase”] with 1 slot .. ..@ .Data:List of 1 .. ..$: int [1:3] 1 1 0 .. ..$ names: chr “AnnotatedDataFrame”

This is similarly the case when glancing at the right secondary side bar, as the respective objects under variables appear to have empty matrices where normally we would have seen the MFI values. Due to flowWorkspace use of pointers, the missing data remains stored on the drive, only being retrieved right before it is required. This reduces the overall RAM utilization. Let’s double check the differences in memory utilization for flowFrame/cytoframe:

::: {.cell}

```
obj_size(flowFrame)
```

:::

3.53 MB

```
obj_size(cytoframe)
```

5.40 kB

And similarly the case for flowSet and cytoset:

```
obj_size(flowSet)
```

20.99 MB

```
obj_size(cytoset)
```

3.88 kB

Despite both R packages having been around for a while, many Bioconductor and GitHub often only implement methods to handle either flowFrames or cytoframes (although newer R packages are now allowing for both). Consequently, as we move forward in the course, it helps

to be aware of which ones we are working with, and have the ability to interconvert between them as needed.

To go from a flowFrame to a cytoframe, we can use the flowFrame_to_cytoframe() function

```
ConvertedToCytoframe <- flowFrame_to_cytoframe(flowFrame)
ConvertedToCytoframe
```

cytoframe object 'file207836ed2889' with 10000 cells and 43 observables: name desc range minRange maxRange $P1TimeNA878.8090878.809P2$ SSC-W NA 4194303.000 0 4194303.000 $P3SSC-HNA4194303.00004194303.000P4$ SSC-A NA 4194303.000 0 4194303.000 $P5FSC-WNA4194303.00004194303.000$.....P39 APC-R700-A CD107a 4192506 -111 4192506 $P40ZombieNIR-AViability4192506-1114192506P41$ APC-Fire 750-A CD27 4192506 -111 4192506 $P42APC-Fire810-ACCR74192506-1114192506P43$ AF-A NA 4194303 -111 4194303 472 keywords are stored in the 'description' slot row names(0):

```
obj_size(ConvertedToCytoframe)
```

5.40 kB

To go from a cytoframe to a flowFrame, we can use the cytoframe_to_flowFrame() function

```
ConvertedToFlowframe <- flowWorkspace::cytoframe_to_flowFrame(cytoframe)
ConvertedToFlowframe
```

flowFrame object '2025_07_26_AB_02_INF052_00_Ctrl.fcs' with 10000 cells and 43 observables: name desc range minRange maxRange $P1TimeNA878.8090878.809P2$ SSC-W NA 4194303.000 0 4194303.000 $P3SSC-HNA4194303.00004194303.000P4$ SSC-A NA 4194303.000 0 4194303.000 $P5FSC-WNA4194303.00004194303.000$.....P39 APC-R700-A CD107a 4192506 -111 4192506 $P40ZombieNIR-AViability4192506-1114192506P41$ APC-Fire 750-A CD27 4192506 -111 4192506 $P42APC-Fire810-ACCR74192506-1114192506P43$ AF-A NA 4194303 -111 4194303 472 keywords are stored in the 'description' slot

This other code gives the same result:

```
ConvertedToFlowframe <- cytoframe_to_flowFrame(cytoframe)
ConvertedToFlowframe
```

```
obj_size(ConvertedToFlowframe)
```

3.53 MB

To go from a flowSet to a cytoSet, we can use the flowSet_to_cytoset() function:

```r
ConvertedToCytoset <- flowSet_to_cytoset(flowSet)
ConvertedToCytoset
```

A cytoset with 6 samples.

column names: Time, SSC-W, SSC-H, SSC-A, FSC-W, FSC-H, FSC-A, SSC-B-W, SSC-B-H, SSC-B-A, BUV395-A, BUV563-A, BUV615-A, BUV661-A, BUV737-A, BUV805-A, Pacific Blue-A, BV480-A, BV570-A, BV605-A, BV650-A, BV711-A, BV750-A, BV786-A, Alexa Fluor 488-A, Spark Blue 550-A, Spark Blue 574-A, RB613-A, RB705-A, RB780-A, PE-A, PE-Dazzle594-A, PE-Cy5-A, PE-Fire 700-A, PE-Fire 744-A, PE-Vio770-A, APC-A, Alexa Fluor 647-A, APC-R700-A, Zombie NIR-A, APC-Fire 750-A, APC-Fire 810-A, AF-A

```r
obj_size(ConvertedToCytoset)
```

3.88 kB

```r
ConvertedToFlowset <- cytoset_to_flowSet(flowSet)
ConvertedToFlowset
```

A flowSet with 6 experiments.

column names(43): Time SSC-W … APC-Fire 810-A AF-A

```r
obj_size(ConvertedToFlowset)
```

20.99 MB

Fortunately, regardless of whether we are using flowFrame/flowSet (RAM) and cytoframe/cytoset (memory pointers), both routes end up converging at the next step, where the underlying .fcs files are passed off to the GatingSet() function.

```r
GatingSet1 <- GatingSet(flowSet)
GatingSet1
```

A GatingSet with 6 samples

```r
class(GatingSet1)
```

[1] “GatingSet” attr(,“package”) [1] “flowWorkspace”

```r
GatingSet2 <- GatingSet(cytoset)
GatingSet2
```

A GatingSet with 6 samples

```r
class(GatingSet2)
```

[1] "GatingSet" attr(,"package") [1] "flowWorkspace"

As we prefaced in the background, beyond the .fcs files themselves, we need infrastructural elements with which to interact with the underlying data, which allows us to organize the various files, transform (scale), compensate (for conventional flow), visualize, derrive statistics, etc. A GatingSet serves as the infrastructural framework that allows us to do this in R.

If we investigate our current GatingSet objects, we won't see much. This will change as we start layering on additional elements. However, rather than try to cram everything into a single week, we will explore in greater depth the individual components over the next three weeks. Instead, for the rest of today, we will work backward, by exploring a GatingSet objecct and what it is capable of doing once fully assembled.

The CytoML R package (also maintained by Mike Jiang) is a sister package to the flowWorkspace. It's main purpose is to permit bringing in existing FlowJo, Diva and Cytobank Workspaces, with all their gates, transformations, etc. into R as fully assembled GatingSet objects. For those who already use one of these commercial softwares, it can be quite useful tool.

Since our goal is to examine a fully assembled GatingSet object, we will be using it today to bring in a FlowJo workspace to R. However, since this is a free Cytometry in R course, and not about to have everyone pay for a license for a one-off topic, in the pre-course Floreada walkthrough I documented how to convert a free Floreada.io workspae into a FlowJo.wsp that can also be used (please note that as of early 2026, some scaling bugs may be present and require troubleshooting).

To get started, let's first attach CytoML to our local environment via the library() call.

```r
library(CytoML)
```

The .wsp files within this week's data where created via Floreada.io. The main difference between the two files is one is a copy of the original that was opened within FlowJo, and subsequently swtiched from logicle to bi-exponential transformation.

We will need to provide the appropiate file path for our desired .wsp file. We can start by identifying which are present using list.files()

```
Folder <- file.path("C:/Users/Macarena/CytometryInR/course/05_GatingSets/data")
FlowJoWsp <- list.files(path = Folder, pattern = ".wsp", full = TRUE)
FlowJoWsp
```

[1] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/FlowJoWSP_Opened-Copy.wsp" [2] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/FlowJoWSP_Un-opened.wsp"

In our case, we will proceed by using str_detect() to select the .wsp that contains the pattern "Opened"

```
ThisWorkspace <- FlowJoWsp[stringr::str_detect(FlowJoWsp, "Opened")]
ThisWorkspace
```

[1] "C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/FlowJoWSP_Opened-Copy.wsp"

With our single .wsp filepath now identified, we can now proceed to set up the intermediate object using open_flowjo_xml()

```
ws <- open_flowjo_xml(ThisWorkspace)
ws
```

File location: C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/FlowJoWSP_Opened-Copy.wsp

Groups in Workspace Name Num.Samples 1 All Samples 6

```
class(ws)
```

[1] "flowjo_workspace" attr(,"package") [1] "CytoML"

Having set up the intermediate flowjo_workspace object, we can attempt to read in the actual data from the .wsp into a GatingSet using the flowjo_to_gatingset() function.

However, due to how I named the original .fcs files ("GROUPNAME" being individual specimens, "TUBENAME" being either Ctrl or SEB), and downsampled to the same number of cells, we will encounter the following error:

```
gs <- flowjo_to_gatingset(ws=ws, name=1, path = Folder)
```

Error: ! Multiple FCS files match sample 00_Ctrl.fcs by filename, event count, and keywords. Candidates are:' C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF052_00_Ctrl.fcs C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF100_00_Ctrl.fcs C:/Users/Macarena/CytometryInR/course/05_GatingSets/data/2025_07_26_AB_02_INF179_00_Ctrl.fcs Please move incorrect files out of this directory or its subdirectories. Show Traceback

```
gs
```

Error: ! object 'gs' not found

As with any error, my first move is to check the help documentation. In this case, my initial response is to see if I can identify an argument that will help differentiate between the names for each specimen.

```
?flowjo_to_gatingset
```

parseWorkspace(obj, …)

S4 method for signature 'flowjo_workspace'

parseWorkspace(obj, …)

flowjo_to_gatingset(ws, name = NULL, subset = list(), execute = TRUE, path = " ", cytoset = NULL, backend_dir = tempdir(), backend = get_default_backend(), includeGates = TRUE, additional.keys $=$"TOT", $additional.sampleID = FALSE, keywords = character(), keywords.source =$ "XML", $keyword.ignore.case = FALSE, extend_val = 0, extend_to = -4000, channel.ignore.case = FALSE, leaf.bool = TRUE, include_emptytree = FALSE, skip_faulty_gate = FALSE, compensation = NULL, transform = TRUE, fcs_file_extension =$ ".fcs", $greedy_match = FALSE, mc.cores = 1, ...)Inthiscase, Ifindthatthe$"$additional.keys$"$argumentwouldlik$additional.keyscharactervector : Thekeywords(parsedfromFCSheader)tobecombined(concatenatedwith$"$.$")w$
(total number of cells) and more keywords can be added to make this GUID.

```
gs <- flowjo_to_gatingset(ws=ws, name=1, path = Folder, additional.keys="GROUPNAME")
gs
```

A GatingSet with 6 samples

```
class(gs)
```

[1] “GatingSet” attr(,“package”) [1] “flowWorkspace”

Especially when working with CytoML, it is often good to have an idea of how long it will take a particular function to run (to better plan how to use our time while waiting, whether to go grab coffee, etc.). There are a couple ways to do so.

One, using the system.time() function from base R, in which we surround whatever line of code we wish to evaluate in {}

```
system.time({flowjo_to_gatingset(ws=ws, name=1, path = Folder, additional.keys="GROUPNAME")}
```

user system elapsed 0.81 0.04 0.89

Alternatively, if we install the bench package, we can use the mark function to evaluate how long it takes on average across numerous iterations.

```
install.packages("bench")
```

```
library(bench)
```

```
mark(Test <- flowjo_to_gatingset(ws=ws, name=1, path = Folder, additional.keys="GROUPNAME"),
```

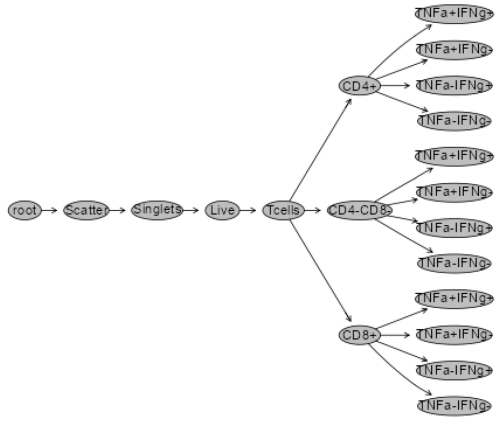
A tibble: 1 × 13

expression min median `itr/sec` mem_alloc `gc/sec` n_itr n_gc total_time result memory time gc
<bch:>
1 “Test <- flowjo_to_gatingset… 859ms 865ms 1.15 13.3KB 0 5 0 4.35s

Now that we have loaded the contents of the FlowJo/Floreada workspace, we can start exploring the various infrastructural capabilities of a GatingSet object.

Let’s start by evaluating whether the manually-drawn gates I drew survived the journey. To do this, I can generate a visual gating treee using the plot() function.

```
plot(gs)
```



We can also retrieve the individual gates and their gaing paths using the gs_get_pop_paths() function.

```
gs_get_pop_paths(gs)
```

[1] "root" "/Scatter"
[3] "/Scatter/Singlets" "/Scatter/Singlets/Live"
[5] "/Scatter/Singlets/Live/Tcells" "/Scatter/Singlets/Live/Tcells/CD4+"
[7] "/Scatter/Singlets/Live/Tcells/CD4+/TNFa+IFNg+" "/Scatter/Singlets/Live/Tcells/CD4+/TNFa+IFNg-"
[9] "/Scatter/Singlets/Live/Tcells/CD4+/TNFa-IFNg+" "/Scatter/Singlets/Live/Tcells/CD4+/TNFa-IFNg-"
[11] "/Scatter/Singlets/Live/Tcells/CD4-CD8-" "/Scatter/Singlets/Live/Tcells/CD4-CD8-/TNFa+IFNg+" [13] "/Scatter/Singlets/Live/Tcells/CD4-CD8-/TNFa+IFNg-" "/Scatter/Singlets/Live/Tcells/CD4-CD8-/TNFa-IFNg+" [15] "/Scatter/Singlets/Live/Tcells/CD4-CD8-/TNFa-IFNg-" "/Scatter/Singlets/Live/Tcells/CD8+"
[17] "/Scatter/Singlets/Live/Tcells/CD8+/TNFa+IFNg+" "/Scatter/Singlets/Live/Tcells/CD8+/TNFa+IFNg-"
[19] "/Scatter/Singlets/Live/Tcells/CD8+/TNFa-IFNg+" "/Scatter/Singlets/Live/Tcells/CD8+/TNFa-IFNg-"

If we wanted to retrieve counts of cells found within the individual gates, we could do so with gs_pop_get_count_fast()

```
Data <- gs_pop_get_count_fast(gs)
head(Data, 5)
```

```
             name                          Population                          Parent Count Pare
           <char>                              <char>                          <char> <int>
```

1: 00_Ctrl.fcs_INF052 /Scatter root 10000 10000 2: 00_Ctrl.fcs_INF052 /Scatter/Singlets /Scatter 9986 10000 3: 00_Ctrl.fcs_INF052 /Scatter/Singlets/Live /Scatter/Singlets 9708 9986 4: 00_Ctrl.fcs_INF052 /Scatter/Singlets/Live/Tcells /Scatter/Singlets/Live 9708 9708 5: 00_Ctrl.fcs_INF052 /Scatter/Singlets/Live/Tcells/CD4+ /Scatter/Singlets/Live/Tcells 6151 9708

Since GatingSets contain multiple .fcs files, we may want to be able to subset them based on metadata for a particular variable. We can check to see current metadata using the pData() function.

```
pData(gs)
```

```
                       name
```

00_Ctrl.fcs_INF052 00_Ctrl.fcs 00_Ctrl.fcs_INF100 00_Ctrl.fcs 00_Ctrl.fcs_INF179 00_Ctrl.fcs 00_SEB.fcs_INF052 00_SEB.fcs 00_SEB.fcs_INF100 00_SEB.fcs 00_SEB.fcs_INF179 00_SEB.fcs

It currently doesn't have much, but we will explore how to change this more over the next few weeks. For now, just know that we could add additional metadata via either a .csv file, or by retrieving additional description keywords from within the .fcs files themselves (as shown below)

```
AlternateGS <- flowjo_to_gatingset(ws=ws, name=1, path = Folder, additional.keys="GROUPNAME"
 keywords=c("$DATE", "$CYT", "GROUPNAME"))
pData(AlternateGS)
```

```
                GROUPNAME   $CYT       $DATE        name
```

00_Ctrl.fcs_INF052 INF052 Aurora 26-Jul-2025 00_Ctrl.fcs 00_Ctrl.fcs_INF100 INF100 Aurora 26-Jul-2025 00_Ctrl.fcs 00_Ctrl.fcs_INF179 INF179 Aurora 26-Jul-2025 00_Ctrl.fcs 00_SEB.fcs_INF052 INF052 Aurora 26-Jul-2025 00_SEB.fcs 00_SEB.fcs_INF100 INF100 Aurora 26-Jul-2025 00_SEB.fcs 00_SEB.fcs_INF179 INF179 Aurora 26-Jul-2025 00_SEB.fcs

Within R, most plots are generated using the ggplot2 package from the tidyverse. The Bioconductor ggcyto R package extends this concept to enable flow cytometry data contained within a GatingSet to be plotted.Gplot2 recently had a major version change, with significant internal changes occuring. As a consequence of these changes, ggcyto functions that relied on the old ggplot2 functions broke and had to be updated. Let's start off by checking what version of both the ggplot2 and ggcyto packages you currently have installed on your computer.

```
packageVersion("ggplot2")
```

[1] '4.0.2'

```
packageVersion("ggcyto")
```

[1] '1.38.1'

The version for ggplot2 is the newest one. However, for ggcyto, there is a newer version, 1.39.3. So, I am going to remove ggcyto first, close Positron an run all chunks again (go to the last chunk and clic in "run above") and install again ggcyto:

```
remove.packages("ggcyto")
```

Removing package from 'C:/Users/Macarena/AppData/Local/R/win-library/4.5' (as 'lib' is unspecified)

```
remotes::install_github("RGLab/ggcyto")
```

When I have reinstalled ggcyto, ggplot2 is also updated, among others CRAN packages, now this are the versions intalled: > packageVersion("ggplot2") [1] '4.0.3' > packageVersion("ggcyto") [1] '1.39.4'

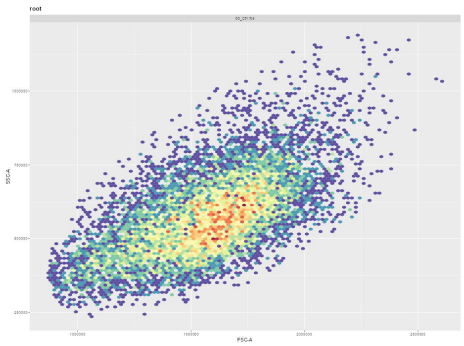
```
library(ggplot2)
library(ggcyto)
```

Loading required package: ncdfFlow

Loading required package: BH

Ggcyto follows the ggplot2 grammar of graphics syntax, which we will learn more extensively next week. For now, lets look at a simple example:

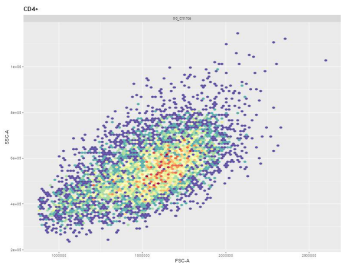
```
ggcyto(gs[1], subset="root", aes(x="FSC-A", y="SSC-A")) + geom_hex(bins=100)
```



The function responsible for plotting is the ggcyto() function. The first argument ("gs[1]") is designating which .fcs file in our GatingSet we are trying to visualize.

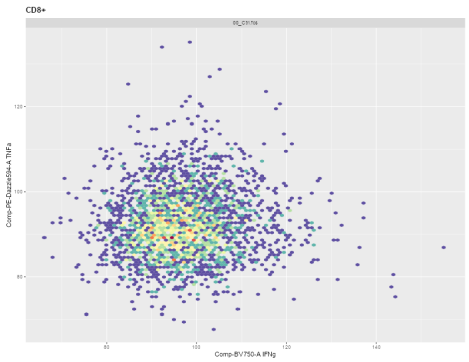
The second argument ("subset") corresponds to which gating node we want to visualize. In this case, when set to "root", we are seeing all cells present in the .fcs file. If we however wanted to visualize the cells within the CD4+ gate, we would swap the value provided to this argument.

```
ggcyto(gs[1], subset="CD4+", aes(x="FSC-A", y="SSC-A")) + geom_hex(bins=100)
```



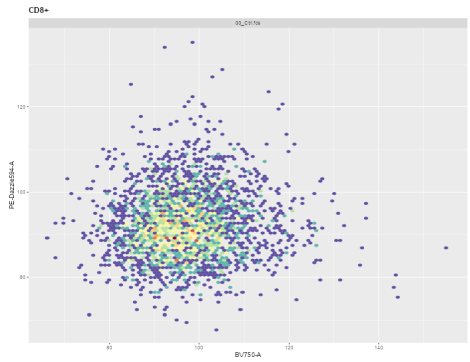
For CD8+

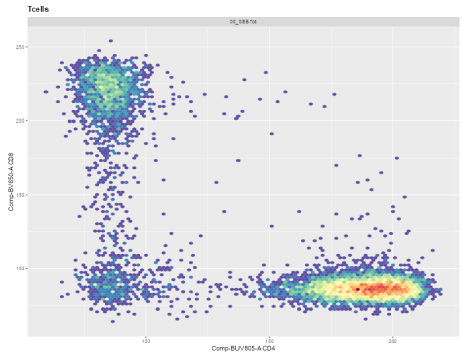
```
ggcyto(gs[1], subset="CD8+", aes(x="IFNg", y="TNFa")) + geom_hex(bins=100)
```



If we didn't remember the marker, we could specify the fluorophore:

```
ggcyto(gs[1], subset="CD8+", aes(x="BV750-A", y="PE-Dazzle594-A")) + geom_hex(bins=100)
```




```
ggcyto(gs[6], subset="Tcells", aes(x="CD4", y="CD8")) + geom_hex(bins=100)
```